

Decision-making tool (time vs profit) for a "personal shopper"

Jin Zhou
zhou.jin3@northeastern.edu
Northeastern University
Seattle, Washington, USA

Abstract

This research project aims to assist personal shoppers in planning and optimizing delivery routes by employing Skyline operator principles to balance two conflicting factors: delivery time and delivery profit. Building upon the original BSL-PSD [2] algorithm, improvements have been implemented to adapt the new algorithm for scenarios with limited product supply in stores. Reorder and timeout mechanisms have been incorporated to address challenges related to path combination uniformity and infinite search deadlocks. Notably, the new algorithm does not significantly reduce the search performance of the original algorithm while providing more route options in the results.

Keywords

Skyline Operator, Route Optimization

ACM Reference Format:

Jin Zhou. 2025. Decision-making tool (time vs profit) for a "personal shopper". In . ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

A *personal shopper* operates as an intermediary agent, executing procurement tasks from customers' itemized shopping lists and generating revenue through strategic markup pricing. The service involves complex procurement processes that present multiple optimization variables requiring systematic analysis.

The core operational framework encompasses two interdependent components:

List fulfillment: This component encompasses the geographical and temporal aspects of the shopping process. Personal shoppers must navigate through multiple retail locations to fulfill customer orders, considering store locations and travel times.

Resale optimization: This component represents the economic dimension of the service. Personal shoppers must strategically select retail locations that offer optimal procurement costs for requested items, as price variations across stores significantly impact potential profit.

These components give rise to two orthogonal decision vectors that govern the multi-objective optimization problem:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

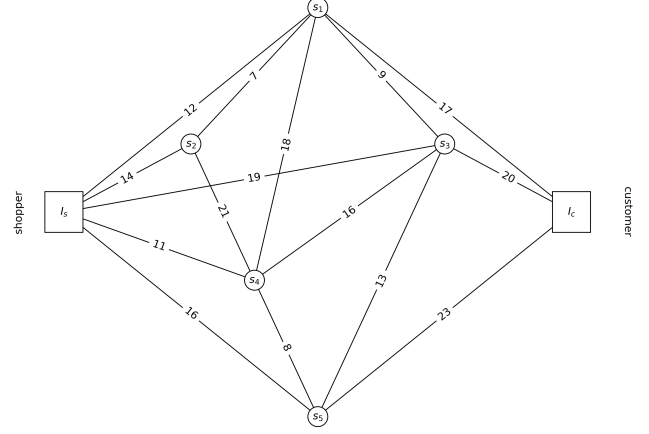


Figure 1: Delivery Network

Time: Determined by the cumulative travel duration along the route, primarily influenced by spatial distances between nodes (current location, stores, and delivery address).

Profit: Calculated as the total resale revenue minus the total procurement cost, where cost variations for identical items across different stores create trade-off opportunities. The absence of direct correlation between procurement costs and geographical proximity creates interesting optimization scenarios.

The relationship between time and profit factors can be illustrated through a system that combines graph and table: an undirected graph $G = (V, E)$, where V represents the set of vertices (including the shopper's starting location, retail stores, and delivery destination) and E represents the set of edges connecting these vertices. As shown in Figure 1, each edge is labeled with the travel time between locations. For the each store's product assortment, pricing, and inventory quantities are recorded in our tables as illustrated in 1 and 2. These tables provide the comprehensive data on product availability and cost variations across different retail locations.

Route travel time and path cost cannot be directly fitted into a linear relationship, as there are many factors affecting product cost such as store type, store location distribution, and others. Therefore, judgments cannot be made based solely on the path travel dimension. Additionally, personal shoppers' path selections depend on their preferences at the time - sometimes shoppers may prefer routes that generate more income despite longer travel times, while other times they may choose paths with less profit to complete orders more quickly. For this purpose, the domination concept from the Skyline operator [3] will be employed to compare

Store	Products Price
s_1	(A, \$9), (B, \$7), (F, \$11)
s_2	(C, \$12), (D, \$9), (E, \$8)
s_3	(C, \$6), (D, \$5), (F, \$8)
s_4	(C, \$10), (D, \$8), (F, \$13)
s_5	(A, \$5), (B, \$8), (E, \$9)

List of products available in the stores depicted in Figure 1. Each pair represents a specific product and the associated unit cost.

Table 1: List of products available and price

Store	Product Quantities
s_1	(A, 15), (B, 20), (F, 8)
s_2	(C, 12), (D, 18), (E, 10)
s_3	(C, 25), (D, 30), (F, 15)
s_4	(C, 18), (D, 22), (F, 7)
s_5	(A, 22), (B, 16), (E, 14)

Inventory quantities of products available in the stores depicted in Figure 1. Each pair represents a specific product and the number of units in stock.

Table 2: Inventory quantities of products

Store	Connected Stores (with travel time)
s_1	(s_2 , 7), (s_3 , 9), (s_4 , 18)
s_2	(s_1 , 7), (s_4 , 21)
s_3	(s_1 , 9), (s_4 , 16), (s_5 , 13)
s_4	(s_1 , 18), (s_2 , 21), (s_3 , 16), (s_5 , 8)
s_5	(s_3 , 13), (s_4 , 8)

Distances between stores. Each entry shows the connected stores and the respective distances.

Table 3: Distances between stores

different paths (specific details will be presented in sections three and four), returning an optimal route set for shoppers to choose from. For example, in the road network 1 and the tracking tables Table 2, Table 1, five paths 4 are generated to satisfy the shopping list, where $\langle R_1, R_2, R_3 \rangle$ are non-dominated paths and $\langle R_4, R_5 \rangle$ are dominated paths. This means that candidate paths in the optimal route set must be superior to other paths in at least one dimension (path time or path cost), but cannot be superior in both dimensions simultaneously 2.

Additional objective constraints include:

Store inventory limits: Ensures all candidate routes exclusively traverse stores with sufficient product quantities to fulfill shopping lists.

Multi-order coordination: Optimizes both procurement sequences and delivery scheduling when handling concurrent shopping lists, preventing resource contention across orders. This necessitates advanced scheduling mechanisms to resolve interdependencies between parallel tasks.

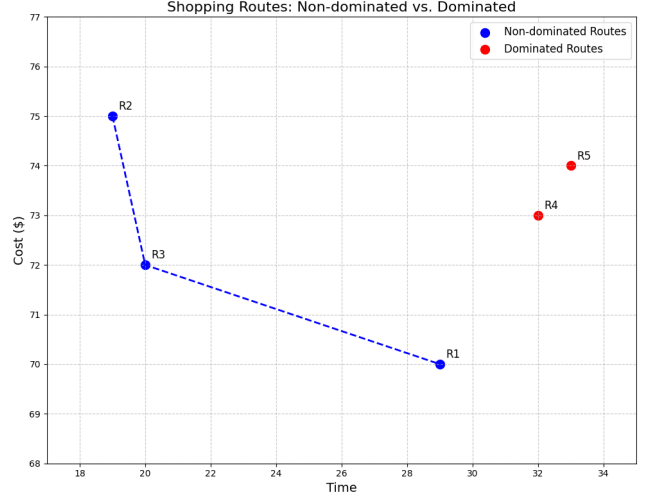


Figure 2: Routes

Route	Time and Cost
R_1	(29, \$70)
R_2	(19, \$75)
R_3	(20, \$72)
R_4	(32, \$73)
R_5	(33, \$74)

Table 4: Candidate Routes

The remainder of this paper is organized as follows: Section 2 reviews relevant literature in route optimization and service scheduling. Section 3 presents the mathematical definitions and explanations of the criteria and conditions. Section 4 presents the mathematical framework and core algorithms. Section 5 details the experimental methodology, including validation experiments and performance evaluations. Finally, Section 6 concludes with conclusions and future research directions.

2 Related Work

Prior research related to optimal route subset generation spans multiple domains. We examine key works in route optimization and spatial search optimization.

2.1 Route Optimization

The Skyline operator [3] provides a fundamental framework for identifying optimal solutions under multiple competing criteria. The core concept relies on the notion of dominance: given two solutions p and q , p dominates q if p is at least as good as q in all dimensions and strictly better in at least one dimension. In database queries, it efficiently reduces the solution space by eliminating all dominated options, retaining only those solutions that are not dominated by any others.

Route optimization research has traditionally centered on finding single optimal paths. [8] proposed an algorithm that maximizes

shopper profits, while [1] concentrates on minimizing delivery network travel time. Both methods inevitably introduce fixed external parameters: Zhen et al. [8] incorporate time constraints and average travel costs to determine locally optimal routes, while Dijkstra's approach solely focuses on finding time-minimal paths. However, these approaches typically optimize for a single objective or use weighted combinations of multiple factors, which may not capture the full spectrum of optimal trade-offs between time and profit.

Our work builds upon the Skyline optimization approach proposed in [2] for route planning and optimal route subset generation. The algorithm BSL [2] searches for all non-dominated route subsets within specified upper and lower bounds. Specifically, the time lower bound is determined by the shortest path to complete the order, while the profit upper bound represents the maximum achievable profit for the order. Starting from the shortest path as the basic route, the algorithm generates new candidate routes by adding or removing intermediate stores. Each new route is then evaluated against the basic route using the dominance principle to determine if it should be included in the optimal subset. Unlike previous approaches that focus on single optimal routes or handle multiple objectives through weighted combinations, our framework preserves the full set of non-dominated routes while considering both temporal efficiency and profit maximization. The resulting optimal route subset provides shoppers with the flexibility to choose routes that best align with their individual preferences and constraints.

2.2 Spatial search optimization

Zhen et al. [8] proposed a domain reduction approach to narrow the search space. Their method defines e as the consumer's current location and r as the cost threshold, using this threshold to eliminate stores with higher procurement costs. However, this approach presents two significant limitations:

First, the threshold r is determined based on average costs, which is inherently problematic as a variable metric. Average costs can fluctuate significantly across different regions, seasons, and market conditions, making it difficult to establish a universally applicable threshold value. Second, their spatial partitioning method does not guarantee effective search space reduction. Even within the partitioned regions, the number of candidate stores may remain excessive, particularly in densely populated urban areas where multiple stores with similar cost structures cluster together.

[7] introduced an effective zone-based partitioning strategy that divides the search space into smaller zones based on predefined directions. Their method implements four distinct zone removal variations: Horizontal Zone Removal (HZR), Vertical Zone Removal (VZR), Right-Sided Zone Removal (RZR), and Left-Sided Zone Removal (LZR). This adaptive approach not only reduces the search space effectively but also maintains solution quality by carefully selecting customers from each zone based on their distance to the zone's geographic centroid. While this zone-based approach effectively reduces computational complexity, it primarily focuses on geographical distance without considering economic factors. Additionally, the fixed directional partitioning may not always capture the natural clustering of profitable store locations.

Chang et al. [6] proposed a novel pruning strategy in MANA (MAze routing under NAometer rules) algorithm that significantly

improves search space reduction. Their method introduces two complementary pruning criteria: length-based pruning and equality-based pruning. In length-based pruning, given two partial paths P and P' reaching the same grid point with min/max lengths of S/L and s'/L' respectively, if $S \geq L'$, path P can be safely eliminated as any of its extensions will be longer than those from P' . Their equality-based pruning further eliminates redundant paths by removing paths with identical length bounds reaching the same point, as these paths will generate equivalent solutions. While this pruning approach effectively reduces computational complexity, it presents two notable limitations. First, the pruning criteria exclusively focus on path length comparisons, failing to consider the economic aspects of route selection. This single-dimensional optimization may lead to suboptimal solutions in scenarios where both spatial and profit factors need to be balanced. Second, the method requires maintaining and updating length bounds for each partial path, introducing additional computational overhead that could impact performance in large-scale route planning scenarios.

More recent work has explored adaptive partitioning strategies. Anwar et al. [2] proposed a two-phase spatial optimization approach. First, they use Minimum Bounding Rectangle (MBR) to enclose all qualifying stores, pickup points, and delivery locations. Then, they apply Quadtree partitioning to recursively divide the space into quadrants until the number of stores in each quadrant falls below a threshold value. They further optimize the search process by assigning scores to quadrants to determine search priorities. While these methods effectively reduce search space, they primarily focus on single-dimensional spatial optimization. When handling multiple orders simultaneously, their algorithm tends to prioritize completing all pickups before making any deliveries, potentially overlooking optimal route combinations that could interleave pickup and delivery operations more efficiently.

3 PRELIMINARIES

In our approach, a city's transportation infrastructure is represented as an undirected graph $G(V, L, T)$, where N contains all junctions and road endpoints in the system, L represents the complete set of road segments connecting these points, and T indicates the traversal times for each connection. For any adjacent locations v_i and v_j within the network, the associated travel duration $t(v_i, v_j) \in T$ corresponds to the time required to move between these points along the connecting road segment.

There are four elements in the model: personal shoppers, customers, stores and shopping lists.

Personal Shopper A personal shopper α is responsible for completing the customer's shopping list, which includes obtaining and delivering all requested items in the shopping list. The personal shopper's position within the road network is denoted by vertex $v_\alpha \in V$.

Customer A customer β seeks to acquire a collection of products and submits an order to a personal shopper in the form of a purchase list that provides the type of product and the quantities needed. The ordered products must be delivered to the customer's location, which is represented by the vertex $v_\beta \in V$ within the network.

Store A store location s_j is located at the intersection $v_{s_j} \in V$, with the collection of all store locations forming the set S . Each

store location offers a different variety of products. Products carry a positive price tag that can vary in different store locations; $p(i, s_j)$ is used to represent the price of the item i in the store location s_j . In addition to the price for each product, $q(i, s_j)$ is used to represent the quantities of the item i in the store s_j .

Shopping List A purchase order ω submitted by β is represented as a set of pairs $\langle p, q \rangle$ where p represents a specific product and q indicates the quantity requested for that particular item.

To facilitate a clear understanding of the algorithms, several additional formal definitions must be established.

Definition 3.1 (Shopping Route). A shopping route θ_i over G is a sequence of stores $\langle s_1^i, \dots, s_n^i \rangle$.

Definition 3.2 (Satisfy Shopping List). A satisfy shopping route $R = \langle s_1, s_2, \dots, s_k \rangle$ must satisfy the following two conditions:

- (1) Product Coverage: For every product $p \in \Omega$, there exists at least one store $s_j \in R$ such that p is available at s_j .

$$PC(\Omega, R) = \forall \langle p, n \rangle \in \Omega, \exists s_j \in R : p \in \text{Products}(s_j)$$

- (2) Quantity Fulfillment: For every product-quantity pair $\langle p, n \rangle$ in the shopping list Ω , the total available quantity of product p across all stores in R must be at least n .

$$QF(\Omega, R) = \forall \langle p, n \rangle \in \Omega, \sum_{s_j \in R} \text{Quantity}(p, s_j) \geq n$$

The satisfy route should be:

$$\text{Satisfy}(R, \Omega) = PC(\Omega, R) \wedge QF(\Omega, R)$$

Definition 3.3 (Shopping Time). Let $R = \langle s_1, s_2, \dots, s_k \rangle$ represent a valid shopping route that satisfies a buyer's purchase order Ω . The travel duration associated with route R , denoted as $ST(R)$, is defined as the time required by the personal shopper to navigate the path starting from position v_α , visiting all store locations according to the sequence specified by R , and concluding at the customer's location v_β :

$$ST(R) = SP(v_\alpha, s_1) + \sum_{j=1}^{k-1} SP(s_j, s_{j+1}) + SP(s_k, v_\beta)$$

where $SP(v_i, v_j)$ represents the minimum travel time along the fastest path connecting vertices v_i and v_j (though alternative transportation cost metrics could be employed).

Definition 3.4 (Shopping Cost). Let Ω be the purchase order submitted by a buyer β and let R be a valid shopping route that satisfies Ω . Then, the purchase expense of route R is defined as follows:

$$SC(R) = \sum_{\langle p, n \rangle \in \Omega} \sum_{s_j \in R} (p(p, s_j) \times Q(p, s_j, R))$$

where $Q(p, s_j, R)$ is a function that returns the quantity of product p purchased from store s_j in the route R . This formulation accommodates the practical scenario where the requested quantity of a single product type may be fulfilled by purchases distributed across multiple stores, allowing for cost optimization through strategic procurement from different store locations.

In the aforementioned definition, scenarios where each store imposes quantity restrictions on available goods have been considered. The resultant collection path must satisfy both the variety of items on the shopping list and the quantity required for each item type. When quantity constraints exist, a personal shopper may purchase the same item from different stores to optimize cost efficiency.

Individual shopping routes can be evaluated along two dimensions: the time required to traverse the path and the cost incurred for purchasing the goods. The concept of domination from skyline operators is employed to balance these two dimensions. Two paths can be defined, R_i and R_j , both satisfying the requirements of the shopping list, and $SC(R)$ and $ST(R)$ can be utilized to represent the cost and time of the shopping route.

Definition 3.5 (Domination). Consider R_i and R_j as two satisfied shopping routes and if R_j dominated by R_i need follows:

$$((SC(R_i) < SC(R_j)) \wedge (ST(R_i) \leq ST(R_j))) \quad (1)$$

$$\vee ((SC(R_i) \leq SC(R_j)) \wedge (ST(R_i) < ST(R_j))) \quad (2)$$

In the final optimal route set, all non-dominated routes should be retained. When route R_j is dominated by route R_i , it is assumed that R_i represents a superior choice under the current circumstances, consequently leading to the elimination of path R_j from consideration.

Symbol	Semantics
$s \in S$	A store in the set of stores S
$R_i = \langle s_1^i, \dots, s_n^i \rangle$	Shopping route
$s_j^i \in R_i$	The j -th store visited by R_i
v_α	Shopper's location
v_β	Customer's delivery location
ω	Shopping list
$ST(R_i)$	Shopping time of route R_i
$SC(R_i)$	Shopping cost of route R_i
$R_i < R_j$	R_i dominates R_j
SC^U	Shopping cost upper bound
R^{ST}	route with minimum shopping time

Table 3: Main elements used throughout the paper.

4 SOLUTION

In this section, a method for solving personal shopper route queries will be proposed. The method is an improvement on the BSL-PSD algorithm, which is a benchmark approach capable of calculating optimal linear skylines. Given shopping lists, BSL-PSD evaluates shopping routes that satisfy the lists in strictly increasing shopping time order, and organizes the construction of skylines accordingly. However, the BSL-PSD algorithm assumes that each store supplies unlimited quantities of goods, which does not align with real-world logic. In this scenario, all goods are supplied in limited quantities, and this constraint has been incorporated into the algorithm. During testing, it was found that the algorithm tends to converge on single-path store ordering, so a path reordering mechanism was added to diversify the paths in the path set. Additionally, when the algorithm was observed adapting to scenarios with limited

store supplies, it was found that it could become trapped in infinite searches without an exit condition. To address this, a timeout mechanism was implemented to terminate the search process.

4.1 Tracking Table

Before describing the algorithm, the three main tables used in the search operations are introduced: the Time table, Product table, and Reserves table.

Time table The time table records the minimum time required to travel between different stores. For example, Table 3.

Product table The product table records the types of goods supplied by different stores and the price of each product in each store. For example, Table 1.

Reserves table The storage table records the inventory quantities of different products across different stores. For example, Table 2.

4.2 Reduce the searching area

Before beginning the search, the scope of the search first needs to be defined. When a shopping list ω is received, the Product table is filtered to identify candidate stores that satisfy ω . The selected stores must contain products that fulfill at least one type of item from the shopping list. Note that this filtering process only considers the types of product, without considering whether the quantity available can meet the demands specified in ω .

4.3 The skyline operator

When comparing different paths to determine whether they can become optimal routes, the principle of domination from The skyline operator is followed, which states that "A point dominates another point if it is as good or better in all dimensions and better in at least one dimension." In this scenario, path elements are influenced by two dimensions: the total time required to complete the path and the total cost of fulfilling the consumer's shopping list.

When two paths are compared, these two dimensions are simultaneously evaluated (the Domination formula described in the previous section determines whether a path is dominated). This means that in the final optimal route set, each shopping route either requires less time to complete the shopping list or incurs a lower cost when completing the shopping list. There will not be a path that is superior in both time and cost dimensions simultaneously. Therefore, all paths in the final optimal route set are non-dominated paths. By applying the principles of the skyline operator, optimal solutions can be provided to personal shoppers under different circumstances.

4.4 Baseline & Upper bound

In addition to the 'reduce searching area' approach mentioned earlier, search baselines and upper bounds are also defined. Two boundaries for optimal routes are defined: 1. the path requiring the minimum time to complete the shopping list, and 2. The path with the minimum cost to complete the shopping list. The algorithm uses these two elements as boundaries for the entire search, representing both the starting point and completion point of the search. The definition is:

Computing R^{ST} finds the shopping route with the shortest shopping time. This requires calculating a variant of the travel planning query, specifically finding the fastest combination of paths connecting the shopper's location, stores, and customer delivery location, while satisfying both the types and quantities in ω (expressed as $Satisfy(R, \omega)$).

Computing SC^U finds the minimum cost expenditure to complete ω . This can be achieved by searching through the product table and reserve table. Here, the focus is no longer on the distances between the shopper, stores, and customer locations, but only on whether the store combinations can satisfy the quantity requirements for each item in ω .

Algorithm will start from the benchmark path R^{ST} and obtain new paths by adding and replacing stores within this path, as will be explained further below. The search continues until algorithm find a path R_i where $SC(R_i) = SC^U$. Once this path is found, the search exits and the results are returned.

4.5 Generate new route

After determining the baseline and upper bound boundaries, new routes need to be generated between these boundaries to perform skyline operator comparisons, thereby selecting a set of non-dominated routes. In simple terms, new candidate paths will be generated by either replacing the last store in the original path or adding a new store at the end.

For instance, let's assume there is currently a path R_i , where the store traversal order is $R_i = \langle s_1^i, \dots, s_n^i \rangle$. The first method is to find the store closest to store s_n^i using the Time table and add it to the end of the original path, generating the first candidate path, expressed as:

$$R_{i'} = R_i \circ s_{n+1} = \langle s_1^i, \dots, s_n^i, s_{n+1} \rangle$$

where $s_{n+1} = \arg \min_{s \in S \setminus R_i} t(s_n^i, s)$, $t(s_n^i, s)$ represents the time interval from store s_n^i to store s , and $S \setminus R_i$ represents the set of all stores not in path R_i .

The second mechanism for generating new candidate paths is to replace the last store s_n^i . The replacement store will be the one closest in distance to store s_n^i , thereby generating a new candidate path, expressed as:

$$R_{i''} = \langle s_1^i, \dots, s_{n-1}^i, s'_n \rangle$$

where $s'_n = \arg \min_{s \in S \setminus R_i} t(s_{n-1}^i, s)$.

In addition, it needs to be proven that the baseline route, R^{ST} , cannot be dominated by newly generated routes.

Poof In our scenario, the mechanism for creating new paths is by adding a store to or replacing the last store in an existing path to generate new candidate paths. Our R^{ST} path is the shortest path that can satisfy the shopping list, which can be viewed as the optimal choice where the distance between each store in the path is minimized under the premise of satisfying the shopping list. When new candidate paths are generated through addition and replacement, they all increase the time needed to complete the shopping list, expressed as:

For adding a new store: $ST(R') = ST(R^{ST}) + t(s_n, s_{n+1}) > T(R^{ST})$.

For replacing with a new store: Given that s_n is the last store and enables the constructed R_i to satisfy the shopping list, and s_n is the store choice that minimizes time consumption when departing from s_{n-1} while satisfying the shopping list, it can be proven that when replacing store s_n , either the shopping list cannot be satisfied or time consumption increases:

$$s_n = \arg \min_{s \in S_{\text{satisfy}}} t(s_{n-1}, s) \Rightarrow \forall s'_n \neq s_n :$$

$$\text{Satisfy}(\langle s_1, \dots, s_{n-1}, s'_n \rangle, \Omega) = \text{False} \vee T(\langle s_1, \dots, s_{n-1}, s'_n \rangle) > T(R^{ST})$$

4.6 Reorder mechanism

In practice, it was found that when generating new non-dominated routes R_{new} , the paths often have strong similarities to the initial R^{ST} . These similarities will be demonstrated in the performance summary. Specifically, the store selections at the beginning of the path and their ordering remain consistent. However, the purpose of this paper is not only to balance the time and cost dimensions but also to provide shoppers with diverse path choices.

It was observed that the path similarity occurs because the method of generating new routes only adds or replaces stores at the end of the path, causing the stores at the beginning to remain fixed during the search process. To address this issue, a new path reordering mechanism is proposed. After generating each new candidate path, the stores involved in the path are rearranged according to the minimum traversal time, allowing the stores at the beginning to change dynamically as well. This can be expressed as:

Let $T_{i,j}$ represent the travel time between s_i and s_j , and $\{s \mid s \in R_{\text{reordered}}\} = \{s \mid s \in R_{\text{new}}\}$

$$R_{\text{reordered}} = \arg \min_R \sum_{i=1}^{n-1} T(R[i], R[i+1])$$

4.7 Timeout mechanism

The timeout mechanism serves as an exit strategy when the search becomes trapped in an endless loop. Through testing, it was found that the algorithm can easily enter into an infinite search state. In this situation, the algorithm continuously generates new candidate paths, but these candidate paths R_i are unable to reach our established upper bound boundary where $SC(R_i) = SC^U$. Furthermore, during this infinite search, the number of routes in the optimal route set remains unchanged (experimental data will be presented in the performance section).

To address this issue, a timeout exit mechanism was defined. When the number of routes in the optimal route set remains unchanged for an extended period, the search behavior exits and the results are returned to the personal shopper. The concept of 'extended period' is defined as a certain number of traversal operations, specifically the number of times a route is popped from the Q queue. Additionally, when the number of routes in the optimal route set changes, the timeout counter is reset.

Let Q be the priority queue of candidate routes, S_{opt} the optimal route set, $|S_{\text{opt}}|$ the number of routes in the optimal route set, count a counter for the number of times a route is popped from Q since the last change in $|S_{\text{opt}}|$, and T_{max} the maximum threshold for count before timeout.

$$\text{Timeout}(\text{count}, |S_{\text{opt}}|, T_{\text{max}}) = \begin{cases} \text{True}, & \text{if } \text{count} \geq T_{\text{max}} \\ \text{False}, & \text{otherwise} \end{cases}$$

And whenever $|S_{\text{opt}}|$ changes: $\text{count} = 0$.

Algorithm 1 Searching

Input: Time table TT , product table TP , reserve table TR , shopper's current location v_α , customer's delivery location v_β , shopping list ω .

Output: Optimal routes set L .

```

1:  $T \leftarrow \text{PruneStores}(TP, \omega)$ 
2:  $LS \leftarrow \emptyset$ 
3:  $SC^U \leftarrow \text{ComputeSC}(T, TP, TR, \omega)$ 
4:  $R^{ST} \leftarrow \text{ComputeST}(T, TT, TP, TR, \omega)$ 
5:  $Q \leftarrow \{R^{ST}\}$ 
6: while  $Q \neq \emptyset$  do
7:    $R = \langle s_{k_1}, \dots, s_{k_{|R|}} \rangle, Q \leftarrow \text{Pop}(Q)$ 
8:    $R = \text{Reorder}(R)$ 
9:   if  $\text{Satisfy}(\omega, R)$  then
10:    if  $SC(R) < SC^U$  then
11:       $L \leftarrow \text{Domination}(L, R)$ 
12:    end if
13:    if  $SC^U = SC(R)$  or  $\text{Timeout}$  then
14:      return  $L$ 
15:    end if
16:    end if
17:     $R^S = \langle s_{k_1}, \dots, s_{k_{|R|}} \rangle \oplus \text{Add}(s_{k_{|R|}}, TT, v_\beta)$ 
18:     $R^P = \langle s_{k_1}, \dots, s_{k_{|R|-1}} \rangle \oplus \text{Switch}(s_{k_{|R|-1}}, TT, v_\beta)$ 
19:     $Q \leftarrow \text{Push}((R^S, R^P), Q)$ 
20:  end while
21: return  $L$ 

```

5 EXPERIMENTAL EVALUATION

5.1 Dataset Prepare

The evaluation of our approach utilized road network and POI datasets from three European cities: Amsterdam, Oslo, and Berlin. Since actual store locations weren't available in these datasets, restaurant—which tend to have dispersed distributions throughout urban areas—were employed as proxy locations to represent store distribution patterns.

The experimental datasets incorporated proxy store locations across Berlin, Oslo, and Amsterdam, visually represented in Figure 3. Table 5 provides quantitative details regarding the network characteristics of each city map, including the total vertex count, edge count, and number of proxy store locations utilized in the analysis.

For the overall experimental environment, a total of 30 different types of products will be provided. The base price of each product will be randomly distributed between \$5-15. Depending on different store types, the price and inventory quantity of each product will vary across stores, changing according to the store type.

Different degrees of product prices, product quantities, and store types also need to be defined to make the environment more closely

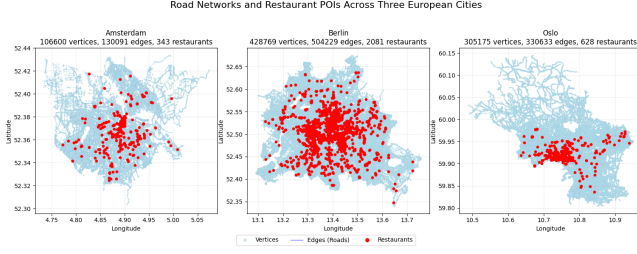


Figure 3: Distribution of stores in each city's road network.

	Amsterdam	Oslo	Berlin
Vertices	106600	305175	428769
Edges	130091	330633	504229
Stores	343	628	2081

Table 5: Metadata for datasets across three European cities.

resemble real conditions. According to the summaries in [4] and [5], store types can be broadly categorized into three sizes.

Small stores represent 60% of our network, carrying only 3-8 products, focusing on bestsellers with a 25% product selection probability. These stores apply a 20% price markup and maintain base inventory levels.

Medium stores make up 30%, offering 8-15 products with greater variety using a 35% selection probability. They provide a 10% price discount and stock twice the base inventory.

Large stores comprise 10%, selling 15-25 products with almost complete product line coverage at a 60% selection probability. They offer 20% discounts and maintain triple the base inventory.

Summarized these in Table Table 6.

	Small Stores	Medium Stores	Large Stores
Network	60%	30%	10%
Range	3-8	8-15	15-25
Probability	25%	35%	60%
Price	+20%	-10%	-20%
Inventory	Base	2× Base	3× Base

Table 6: Distribution Network Store Classification Metadata

Finally, the average optimal performance gap, processing time, and number of paths will be reported. In the experiments, shoppers and customer locations will be randomly selected, and shopping lists of sizes 5, 10, and 15 will be randomly generated. To meet the experimental objectives, the demand quantity of 1-3 types of products can be increased, thereby satisfying situations where multiple stores need to be visited for the same product. All experiments are run on a MacBook Pro with M1 chip, with CPU speed of 3228 MHz and memory of 32GB.

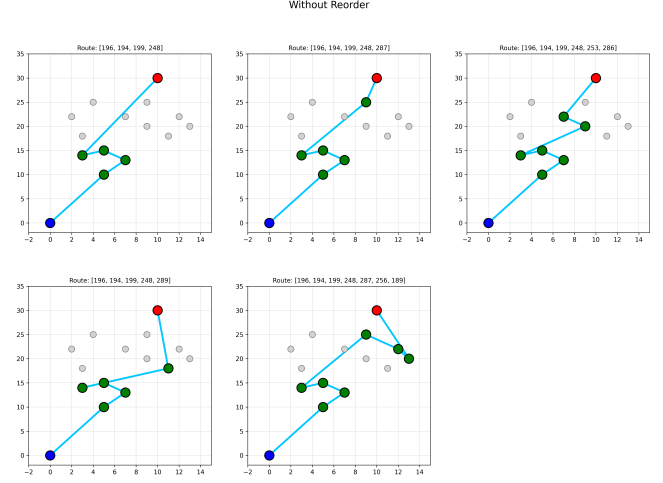


Figure 4: Without Reorder

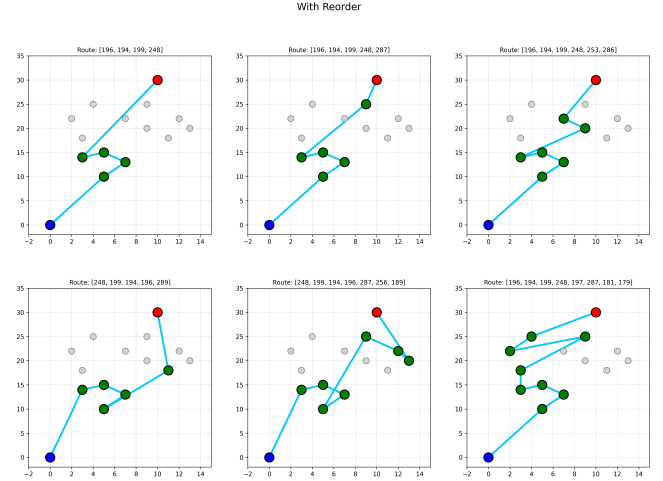


Figure 5: With Reorder

5.2 Prove Experiment

In this subsection, two remaining issues previously mentioned in the solution chapter will be addressed: one concerning path diversification, and the other regarding the problem of falling into endless traversal searches.

5.2.1 Route Diversification. In this experiment, the shopping list size was 8, and search algorithms both with and without the reorder mechanism were tested to compare the differences between the optimal paths returned by each approach. To avoid excessive data, only the network from the city of Amsterdam was selected for this experiment. The experimental data is shown in Figure 4 and Figure 5.

First, as shown in Figure 4, It can be observed that in the experiment without using the reorder mechanism, all paths maintained the same route at the beginning, specifically preserving the order

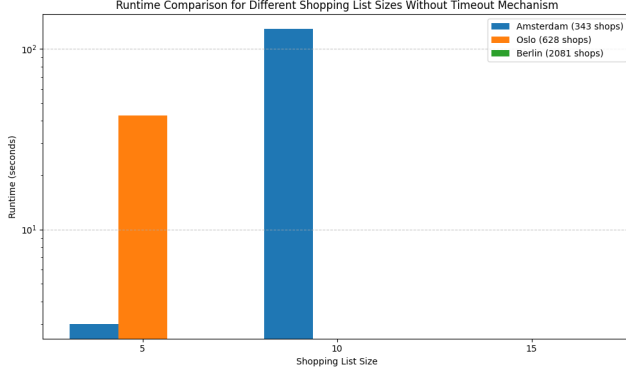


Figure 6: Searching Time Compare

$R = \langle s_{196}, s_{194}, s_{199} \rangle$. This indicates that all paths must pass through these stores in this fixed sequence.

In contrast, our experimental data using the reorder mechanism Figure 5 reveals that not only was s_{248} added to the beginning of the path and the initial ordering was shuffled, but an additional path R_6 was also generated. This demonstrates that the incorporation of the reordering mechanism leads to greater path diversity, thereby providing more options for the personal shopper to choose from.

5.2.2 Timeout Mechanism. In this experiment, the search time for three different shopping list sizes (5, 10, and 15) was compared without using a timeout mechanism. The absence of a timeout mechanism means that the search program only exits when it finds the path with the minimum cost. Under these experimental conditions, the search times across three cities were compared. The experimental data is shown in 6.

From the experimental data, it can be determined that when conducting searches in Amsterdam and Oslo, the path with minimum cost was found when the shopping list size was 5, with Amsterdam's search speed being faster than Oslo's. However, under the same conditions, Berlin could not find the minimum cost path within a reasonable time frame. This can be attributed to the number of stores affecting search efficiency: Amsterdam has only 343 stores, Oslo has 628, while Berlin has 2,081 stores.

Furthermore, when the shopping list size was 10 or 15, all three city maps had difficulty finding the minimum cost path. Analysis suggests that when considering the limited availability of products in each store, the number of possible minimum cost paths decreases. Despite implementing the reorder mechanism to increase path diversity, it remains extremely challenging to find the minimum cost path, indicating that the regions covered by our path transformations cannot reach this optimal path.

5.3 Evaluate Experiment

5.3.1 Limited vs. Unlimited Supply. First, the search efficiency of the algorithm was compared under two different scenarios: unlimited product supply and limited product supply. The comparison factors included: 1. The cost of finding the optimal route, specifically the search cost for finding the path with the shortest time and the minimum cost value. 2. The time consumption during the

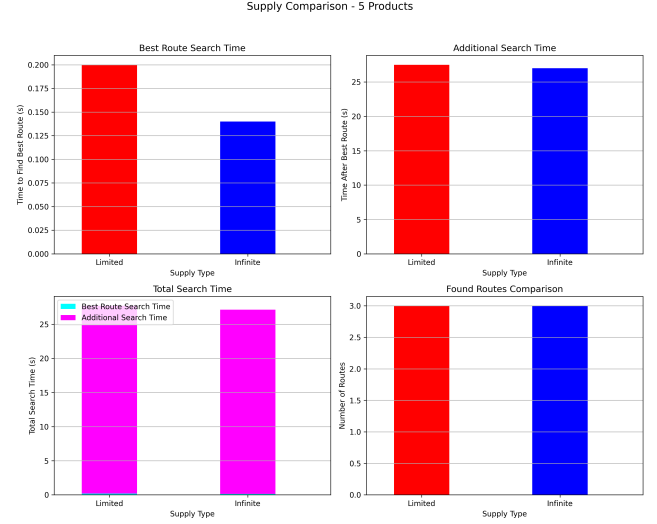


Figure 7: Supply Comparison - 5 Products

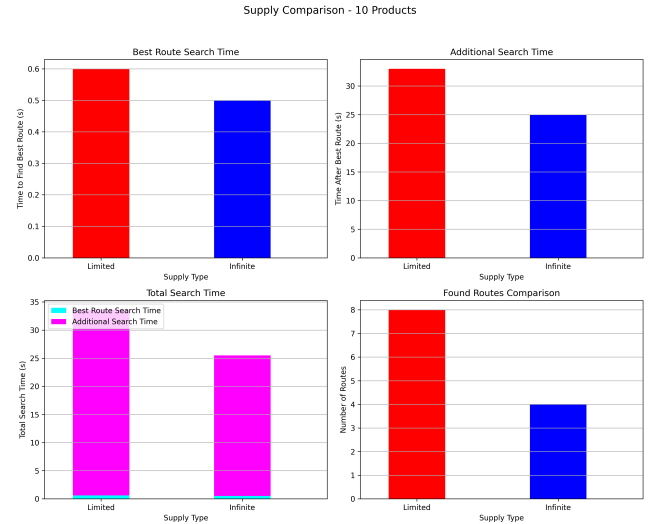


Figure 8: Supply Comparison - 10 Products

search process, excluding the two search boundaries, which includes the time spent generating new paths and performing skyline operations. 3. The total search time consumption. 4. The number of optimal routes ultimately generated.

In the experiment, the road network of Amsterdam (AMS) was used and consistency in the shopping list was ensured, meaning that under both comparison environments, the types of products in the shopping list and their required quantities were identical, and the types of products offered by stores were also consistent. The only difference was the quantity of products provided by each store. Our experimental data is shown in Table 7, Table 8, Table 9.

According to the experimental results: 1. When comparing the search time for finding boundaries, although there is a gap in search

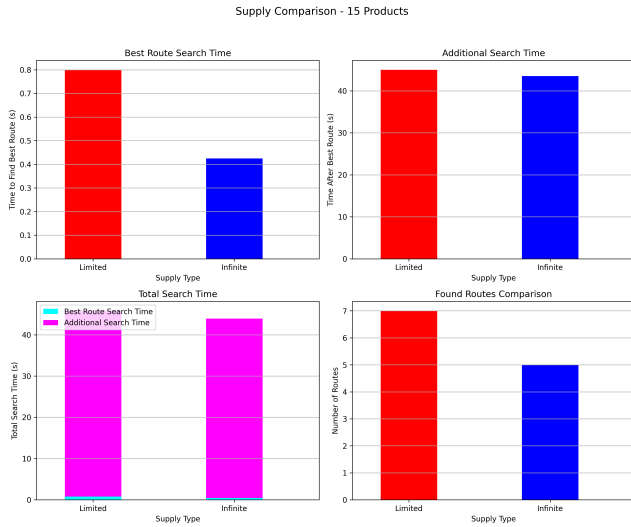


Figure 9: Supply Comparison - 15 Products

time between limited and unlimited product supply scenarios, with the unlimited supply scenario finding boundaries more quickly, the difference is only in milliseconds and is considered acceptable. 2. In terms of search time consumption, it can observe that search time increases with the number of optimal routes, which is related to our timeout mechanism; when the number of optimal routes changes, the timeout counter resets. Simultaneously, it can be seen that when the shopping list size is 5, the number of optimal routes is consistent between both scenarios, and their search times are very close, approximately equal. 3. From the observation of the number of optimal routes, the algorithm can discover more optimal routes in the limited supply environment.

The reason for these differences is that in a limited product supply environment, the selection of stores along the path becomes more complex. This is because a single store may not satisfy the quantity requirement for a product in the shopping list, resulting in situations where the same product must be purchased from multiple stores. This increases the initial boundary search time as multiple scenarios need to be matched, and leads to the discovery of more optimal routes in the limited supply scenario.

5.3.2 City Search Comparison. In this experiment, we compared the search performance across three cities with three different shopping list sizes. Our experimental data is shown in Figure 10.

The experiments indicate that despite implementing a timeout mechanism, search time still increases with the number of stores in the map.

6 CONCLUSION & Future Work

Conclusion This research focused on how to provide an optimal route set using the skyline operator for personal shoppers to choose from, even when store products are subject to quantity limitations, while ensuring that search performance does not significantly deteriorate. During the research, two mechanisms—reorder and timeout—were introduced to address the issues of path uniformity and

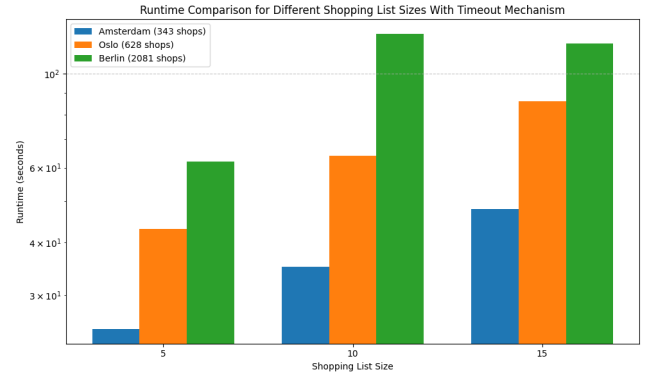


Figure 10: City Comparison

endless searching that were identified. The final experimental results demonstrate that our search performance is within 25% of the unlimited supply scenario, and our algorithm can provide a greater number of optimal routes to choose from.

Future Work Due to time constraints, this paper does not provide solutions for multi-order routing, so we leave this research topic for future researchers. The suggestion is that the core considerations for multi-order problems should be the delivery sequence between orders and the sequence of completing order purchase actions. This could be determined by comparing product repetition between orders to establish product purchase priorities, and then using these priorities to determine order delivery priorities. Additionally, for the algorithm presented in this paper, the timeout mechanism needs improvement. Our additional testing has shown that the threshold value in the timeout mechanism varies with order size and the number of stores in the network, requiring a more adaptive method to address the infinite search problem.

Finally, I would like to share my understanding of this research: the difficulty of this research is not as simple as it may appear. Various problems arise during experimentation, and while I have addressed the issues of path uniformity and endless searching here, new problems may emerge in the future. Regarding the significance of this research, I believe it can help others by enabling people to maintain agency in today's digital society.

References

- [1] Samaher Adnan, Enas Wahab Abood, and Wafaa Abdulmuhsin. 2020. The multi-point delivery problem: Shortest Path Algorithm for Real Roads Network using Dijkstra. *Journal of Physics: Conference Series* 1530, 1 (May 2020), 012040. doi:10.1088/1742-6596/1530/1/012040
- [2] Samiul Anwar, Francesco Lettich, and Mario A. Nascimento. 2020. Towards A Personal Shopper's Dilemma: Time vs Cost. doi:10.48550/arXiv.2008.11705 arXiv:2008.11705 [cs].
- [3] S. Borzsony, D. Kossmann, and K. Stocker. 2001. The Skyline operator. In *Proceedings 17th International Conference on Data Engineering*. IEEE Comput. Soc, Heidelberg, Germany, 421–430. doi:10.1109/ICDE.2001.914855
- [4] Ailing Cai, Jun Wang, Jingxian Li, and Yihang Du. 2018. Spatial Characteristics of Retail Stores in Multiple-Size of Cities in the Central Region of China. *Acta Scientiarum Naturalium Universitatis Pekinensis* 54, 5 (2018), 1114–1122.
- [5] Caitlin E. Caspi, Jennifer E. Pelletier, Lisa J. Harnack, Darin J. Erickson, Kathleen Lenk, and Melissa N. Laska. 2017. Pricing of Staple Foods at Supermarkets versus Small Food Stores. *International Journal of Environmental Research and Public Health* 14, 8 (15 Aug 2017), 915. doi:10.3390/ijerph14080915
- [6] Fong-Yuan Chang, Ren-Song Tsay, Wai-Kei Mak, and Sheng-Hsiung Chen. 2013. MANA: A Shortest Path Maze Algorithm Under Separation and Minimum Length

- NAnometer Rules. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 10 (Oct. 2013), 1557–1568. doi:10.1109/TCAD.2013.2265878
- [7] Uğur Emeç, Bülent Çatay, and Burcin Bozkaya. 2016. An Adaptive Large Neighborhood Search for an E-grocery Delivery Routing Problem. *Computers & Operations Research* 69 (May 2016), 109–125. doi:10.1016/j.cor.2015.11.008
- [8] Lu Zhen, Xueting He, Huiwen Wang, Gilbert Laporte, and Zheyi Tan. 2022. Decision models for personal shopper platform operations optimization. *Transportation Research Part C: Emerging Technologies* 142 (Sept. 2022), 103782. doi:10.1016/j.trc.2022.103782